Information Caution Tip Important

Alertoni

Version 1.0.0

<*> <*>

About

Alertoni is a package that introduces various callouts types with three different styles. In addition the package supports custom callout types, custom styling and switching icon sets.

<*> <*>

```
#import "@preview/alertoni:1.0.0" as at

#at.callout(
  type: "important",
  title: [Kitty!],
  [
    #image("./images/little-kitty.jpg")
  ]
)
```

Kitty!

Photo by Kote Puerto on Unsplash

Contents

I \ Installation

1. Install Tabler.io font

- Tabler.io Icons (Webpage Link or CDN JsDelivr Page[1])

> **Important**
> At default settings, this font **is required**!

2. Import the package in three different ways

- Via Typst Universe

```
#import "@preview/alertoni:1.0.0" as at
```

- Downloading or cloning the repository `"https://codeberg.org/joelvonrotz/typst-alertoni"` and then including the `exports.typ` file.

```
#import "./path/to/typst-alertoni/exports.typ" as at
```

- Via `@local` namespace

```
#import "@local/alertoni:1.0.0" as at
```

II \ Usage

II.1 Callout Styles

Styles in the context of *Alertoni* is how the callout look, so the drawing style. The package comes with three predefined styles and allows for user defined styles.

II.1.a Predefined Styles

The three predefined styles are `"minimal"`, `"compact"`, `"quarto"`. The last style `"quarto"` is inspired by (aka. almost identical copy of) Quarto's callout blocks.

```
#at.callout(
  style: "minimal", [Style `"minimal"`]
)
```

> **Information**
> Style `"minimal"`

```
#at.callout(
  style: "quarto", [Style `"quarto"`]
)
```

> **Information**
> Style `"quarto"`

```
#at.callout(
  style: "compact", [Style `"compact"`]
) -- it is also an inline style.
```

Information Style `"compact"` – it is also an inline style.

[1] You can build the font yourself, the building scripts are available at https://github.com/tabler/tabler-icons. Thus I believe you should have access to these files as well. **But support the icon designers if you can!**

II.1.b Creating & Using Custom Callout Styles

To allow for a more individual customization of the style, a custom callout style can be defined. To «assign» a custom style, a function in the shape of

```
(title, icon, content, paint, height, width) => {...}
```

is passed onto the style parameter of the #callout function.

```
#let my-style(title,_,body,paint,_,_) = {
  text(paint, strong[#title ]) + [#body]
}

#at.callout(style: my-style, [
  about a new style!
])
```

Information about a new style!

Global Configuration?
The function is only applied to the callout function it was passed into. To «globally» set a style, see Tips & Tricks – Global Configuration.

II.2 Callout Types

Callout types define the kind of callouts. As with the styles, the package comes with various predefined types and allows for user defined ones to be added.

II.2.a Predefined Types

```
#at.callout(type: "info",      [])
#at.callout(type: "warning",   [])
#at.callout(type: "important", [])
#at.callout(type: "caution",   [])
#at.callout(type: "tip",       [])
#at.callout(type: "correct",   [])
#at.callout(type: "incorrect", [])
#at.callout(type: "example",   [])
```

Information

Warning

Important

Caution

Tip

Correct

Incorrect

Example

II.2.b Creating & Using Custom Callout Types

To add custom callout types, the function #new-type() is used. Calling this function places the configuration as a state at the call location, meaning you won't be able to use the type BEFORE the call. I recommend setting the types before the overall content!

Each callout type has an «ID» associated to a couple of configurations: `color`, `icon` and `placeholder`. The term *placeholder* is used, because the title of the callout is using a placeholder title when no title is specified!

Now let's create a callout type `"pizza"`:

```
#at.new-type(
  name: "pizza",
  color: red,
  icon: [🍕],
  placeholder: "Pizza Time!"
)
```

With the type added, you can now create callouts and use "pizza" as type to use it.

```
#at.callout(type: "pizza")[#lorem(10)]
```

🍕 **Pizza Time!**
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Important
#new-type will overwrite existing entries.

II.3 Language Context

The predefined types and, if configured, user defined types can have language specific titles. These get automatically selected when setting the respective language via `#set text(lang: "...")`.

Caution
If no title is found, #callout will throw an error, unless a `fallback` entry exists!

```
#let type-name = "pizza"
#let paint = red
#let icon = [🍕]
#let placeholder = (
  fallback: "Pizza Time?" // required if other languages besides the ones below are used
  en: "Pizza Time!",
  de: "Pizza Zeit!"
)

#at.new-type(
  name: type-name,
  color: paint,
  icon: icon,
  placeholder: placeholder
)
```

With the type added, you can now create callouts and use "pizza" as type to use it. Since the placeholder has translations, the callout function will try to select the respective one.

```
#set text(lang: "en")
#at.callout(type: "pizza")[#lorem(6)]

#set text(lang: "de")
#at.callout(type: "pizza", [#lorem(6)])

#set text(lang: "es")
#at.callout(type: "pizza", lorem(6))
```

🍕 **Pizza Time!**
Lorem ipsum dolor sit amet, consectetur.

🍕 **Pizza Zeit!**
Lorem ipsum dolor sit amet, consectetur.

🍕 **Pizza Time?**
Lorem ipsum dolor sit amet, consectetur.

II.4 Modifying the Icon Set

Per default *Alertoni* uses the package tableau-icons for drawing the icons in the callout. This is internally done via a lookup table (LUT), where each style is assigned an icon as content. This of course can be changed via the #set-icons function.

```
#let icon-set = (
  "info": [*i*],
  "warning": [*!*],
  "important": [*!!!*],
)

#at.set-icons(icon-set)

#at.callout(type: "info", [])
#at.callout(type: "warning", [])
#at.callout(type: "important", [])
```

i Information

! Warning

!!! Important

ⓘ Information
When setting icons, existing entries in the internal dictionary are updated and new entries are appended.

The #set-icons function also supports reseting the icons back to default by passing auto. This also clears all user defined icons.

```
#at.callout(type: "info", [])
#at.set-icons(auto)
#at.callout(type: "info", [])
```

i Information

ⓘ Information

III \ Tips & Tricks

III.1 Global Configuration

To globally change the callout style, default type, you can overload the #callout function using #at.callout.with(..):

```
#let quarto-tip = at.callout.with(style:
"quarto", type: "tip")

#quarto-tip[Hello!]
```

Tip
Hello!

Caution
If you have a multi file project, place it in a «preamble» file, that gets imported in every subdocument that requires callouts.

Important
The reason for this kind of implementation is, that I don't want to use custom type packages like elembic (for now), as I'm waiting for Typst to officially support custom types.

It's not the best solution, but a useable workaround until Typst supports custom types. Once these are added, the callout configuration will be configurable via (hopefully):

```
#set at.callout(style: my-style, type: "tip")
```

III.2 Writing Shortcut

To make writing a bit easier or flow nicer, the #callout function has a little trick up its sleeve using positonal arguments. By passing an additional positional argument, the first one will be used for the title and the second one as body.

# of pos()	Effect
1	[0]: Body of callout
2	[0]: Title of callout [1]: Body of callout
otherwise	Error is thrown

The argument switchup was done to make it easier to read the code line. When reading callouts, usually title is read first, then body.

```
#at.callout[No shorthand version]

#at.callout[Shorthand][Version!]
```

Information
No shorthand version

Shorthand
Version!

III.3 Function Shortcuts

Another handy use of `#at.callout.with(..)` is to create functions for each callout type. For a `#info` callout function, you'd declare

```
#let info = at.callout.with(type:"info")

#info(title: [Lorem Ipsum!])[#lorem(10)]
```

Lorem Ipsum!
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

Function shortcuts were not implemented in the package, as it might conflict with functions in your document. But to make life a bit easier, below are all the predefined types as functions you can copy into your project!

```
#import "@preview/alertoni:1.0.0" as at

#let info = at.callout.with(type: "info")
#let warning = at.callout.with(type: "warning")
#let important = at.callout.with(type: "important")
#let caution = at.callout.with(type: "caution")
#let tip = at.callout.with(type: "tip")
#let correct = at.callout.with(type: "correct")
#let incorrect = at.callout.with(type: "incorrect")
#let example = at.callout.with(type: "example")
```

IV \ API Reference

IV.1 callout

```
#at.callout(title: [Title], [Content])
```

Title
Content

```
#at.callout([Content with automatic Title])
```

Information
Content with automatic Title

IV.1.a Parameters

```
callout(
  type: string,
  style: string function,
  title: auto none ,
  width: auto relative fraction,
  height: auto relative fraction,
  icon: content auto,
  color: color auto,
  ..content
) -> content
```

type `string`

Type of the callout. Predefined styles are `"info"`, `"warning"`, `"important"`, `"tip"`, `"caution"`, `"correct"`, `"incorrect"`, `"example"`.

If user defined types have been set via #new-type, can also contain the respective type name.

```
#at.callout(type: "info", lorem(5))

#at.callout(type: "caution", lorem(5))

#at.callout(type: "tip", lorem(5))
```

Information
Lorem ipsum dolor sit amet.

Caution
Lorem ipsum dolor sit amet.

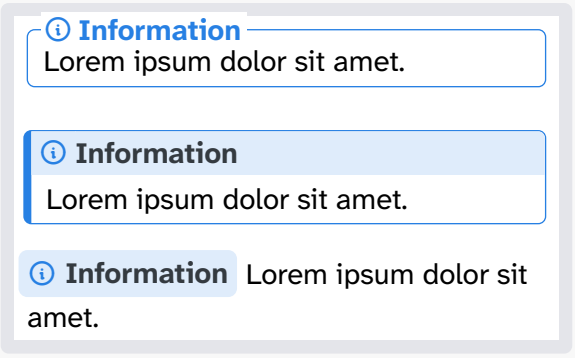
Tip
Lorem ipsum dolor sit amet.

Default: `"info"`

style `string` or `function`

Rendering style of the callout. Predefined styles are `"minimal"`, `"compact"` and `"minimal"`.

```
#at.callout(style: "minimal", lorem(5))
#at.callout(style: "quarto", lorem(5))
#at.callout(style: "compact", lorem(5))
```

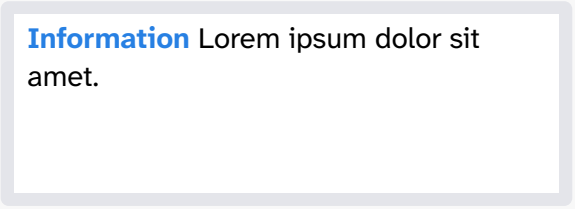


Passing a function in the shape of

```
(title, icon, content, paint, height, width) => {...}
```

allows for a user defined style.

```
#at.callout(style:
  (title,_,body,paint,_,_) => {
    [*#text(paint, title)* #body]
  },
  lorem(5)
)
```



Default: `"minimal"`

title `auto` or `none` or

Title of the callout.

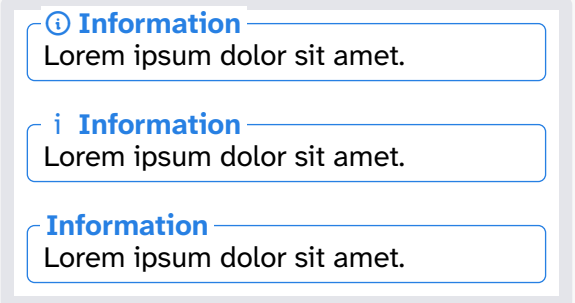
When set to `auto` the title is automatically chosen from the (user- or pre-) assigned placeholder title.

When set to `none`, hides title. If set to anything else, callout title is overwritten. **This applies but is not limited to the predefined styles.**

```
#at.callout(icon: auto, lorem(5))

#at.callout(icon: [i], lorem(5))


#at.callout(icon: none, lorem(5))
```



Default: `auto`

width `auto` or `relative` or `fraction`

Width of the callout, but depends on the style. Style `"compact"` for example ignores this parameter.

Default: `100%`

height `auto` or `relative` or `fraction`

Height of the callout, but depends on the style. Style `"compact"` for example ignores this parameter.

Default: `auto`

icon `content` or `auto`

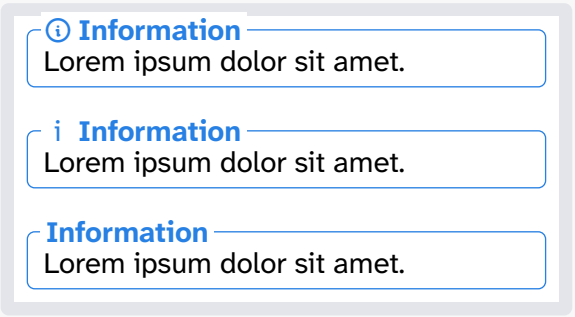
The icon of the callout. Per default the icon is automatically selected from the respective type configuration. Passing anything that can render as content, overwrites the «default» icon.

When set to `none`, does not render any icons. **This applies but is not limited to the predefined styles.**

```
#at.callout(icon: auto, lorem(5))

#at.callout(icon: [i], lorem(5))


#at.callout(icon: none, lorem(5))
```



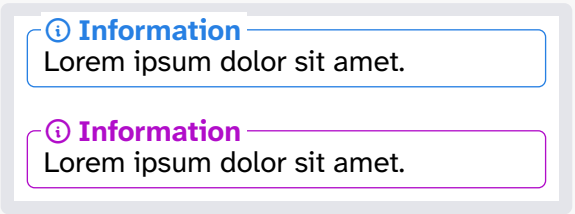
Default: `auto`

color `color` or `auto`

The color of the callout. Per default the color is automatically selected from the respective type configuration. Passing a color, overwrites the «default» color.

```
#at.callout(color: auto, lorem(5))


#at.callout(color: purple, lorem(5))
```

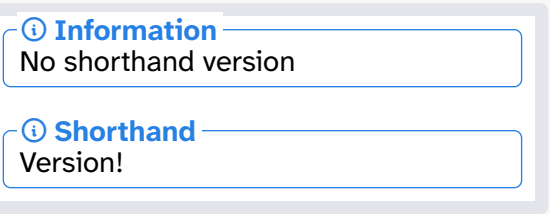


Default: `auto`

..content

The body of the calloutm, **IF** only one position item is passed. If two position items are passed, the first item is the callout, the second one is the callout body.

```
#at.callout[No shorthand version]
#at.callout[Shorthand][Version!]
```



This improves writing callouts slightly, as in doesn't break writing flow as much.

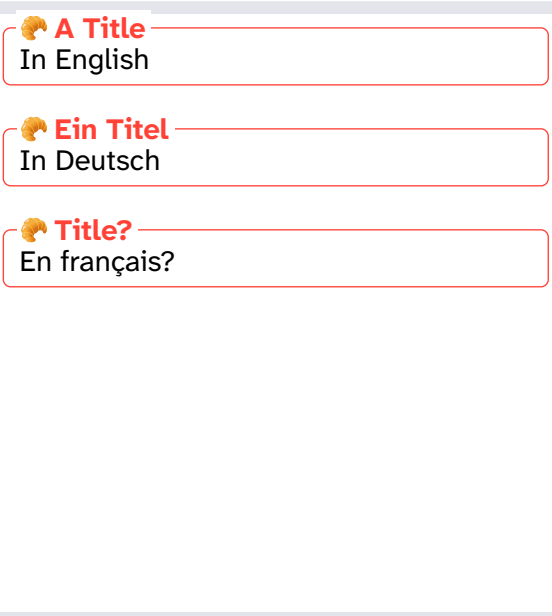
IV.2 new-type

```
#at.new-type(
  name: "test",
  color: red,
  icon: text(0.8em)[#emoji.croissant],
  placeholder: (
    fallback: "Title?",
    en: "A Title",
    de: "Ein Titel"
  )
)

#set text(lang: "en")
#at.callout(type: "test", [In English])

#set text(lang: "de")
#at.callout(type: "test", [In Deutsch])

#set text(lang: "fr")
#at.callout(type: "test", [En français?])
```



IV.2.a Parameters

```
new-type(
  name: str,
  color: color,
  icon: content str none,
  placeholder: dictionary content str
) -> none
```

name `str`

Name of the new callout type which is also to be used in the `#callout` function.

Default: none

color color

The color of the callout. Depends on the style on how it is used.

Default: none

icon content or str or none

The icon to be associated with the callout. Everything not none will be added to the icon-set via #set-icons(). Passing none skips this step.

Default: none

placeholder dictionary or content or str

The fallback/automatic title used when no title is given when calling #callout. Passing a dictionary with the key-value shape of (<lang> : <title>) allows for language dependent titles (for the languages in the dictionary).

> **Important**
> If a dictionary is used, #callout will throw an error, if the respective title cannot be found.
>
> This can be circumvented via an "fallback" entry!

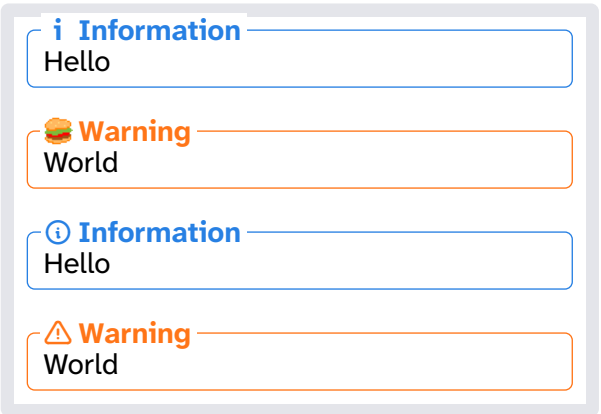
Default: none

IV.3 set-icons

Configures the icons by passing a dictionary, which assigns a content value to the respective callout type.

```
#let new-set = (
  "info": [*i*],
  "warning": burger-image(width: 100%)
)
#at.set-icons(new-set)
#at.callout(type: "info", [Hello])
#at.callout(type: "warning", [World])

#at.set-icons(auto)
#at.callout(type: "info", [Hello])
#at.callout(type: "warning", [World])
```



IV.3.a Parameters

set-icons(new-set: dictionary auto none)

new-set `dictionary` or `auto` or `none`

Passing a dictionary in the format of `(<type> : <icon>, ...)`, allows you to add or overwrite the icon assignments for specific callout types.

Passing `auto`, resets all the default types.
Passing `none`, resets all the default types **and** clears entries of user defined types.

V \ Changelog

1.0.0 \ Initial Release

- Added functions #callout, #new-type, #set-icons with documentation!